MOVIEFLIX

Intelligent Movie Search System

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

Hemaswi:2520030069

Simi Ayana:2520030254

Rushvini:2520030542

Team Number:8

Section:

Under the Guidance of

Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Title of the Project"** is a record of Bonafide work of **G.Hemaswi – 2520030069, Rushvini – 2520030542, Simi Ayana– 252030254** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

G.Hemaswi – 2520030069

Simi Ayana – 2520030254

Rushvini – 2520030542

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled **"Title of the Project"** is a bonafide work done and submitted by **G.Hemaswi – 2520030069, Rushvini – 2520030542, , Simi Ayana – 2520030254** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

Description in Times New Roman font 12 size with two to three paragraphs thanking guide Dr. S. Vinay Kumar, Associate Professor, Dept. of CSE and Dr. P. Venkateshwara Rao, Associate Professor, HOD CSE

G.Hemaswi – 2520030069

Simi Ayana – 2520030254

Rushvini – 2520030542

Abstract

MovieFlix is a Java-based console application that demonstrates movie catalogue management and algorithmic search over a CSV dataset. The core system stores movie records containing an identifier, title, language, genre, release year, director, and description. It supports displaying records, exact-title lookup using a HashMap, title lookup using Binary Search after sorting, keyword matching using Knuth–Morris–Pratt (KMP), approximate title matching using Levenshtein distance, sorting by title or release year using Merge Sort, filtering by language and genre, and create, update, and delete operations. CSV loading and saving provide simple persistence between runs.

The application also includes an Advanced DSA-3 Lab menu organised into modules M1–M6. These modules demonstrate or discuss system/query routing, string algorithms, dynamic programming, network flow, NP-completeness and approximation, and randomized/parallel algorithm concepts. The implemented demonstrations include Z-function, Rabin–Karp, Aho–Corasick, suffix-array/LCP construction, edit-distance and alignment DP, matrix-chain DP, bitmask TSP DP, Ford–Fulkerson, Edmonds–Karp, Dinic, a vertex-cover approximation, Miller–Rabin primality testing, randomized QuickSort, and reservoir sampling. Some topics—such as min-cost max-flow, bipartite matching, randomized hashing, and parallel execution—are explanatory previews rather than complete implementations.

The project illustrates how algorithm choice affects search performance and how a single console application can bring together data structures, sorting, string processing, dynamic programming, graph algorithms, approximation, and randomized techniques. The current implementation is a learning prototype; its limitations and potential corrections are documented so that results are not overstated.

Table of Contents

			Page No.
	Acknowledgement		
	Abstract		
	Table of Contents		
	List of Figures		
1	Introduction	1.1 Overview 1.2 Problem statement 1.3 Scope of the project 1.4 Objectives	5-15
2	System Requirements	2.1 Functional Requirements 2.2 Non-Functional Requirements	16-21
3	System Design	3.1 High-level Architecture Diagram 3.2 Description of each layer	33-36
4	Implementation	4.1 Module-wise implementation 4.2 Front-end logic (UI rendering, state management) 4.3 API integration 4.4 …….	38-47
5	Results		56-64
6	Challenges & Limitations	6.1 Issues faced during development 6.2 Solutions applied 6.3 Limitations of the System	74-82
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills learned during development	89-97
	References	- Books, tutorials, APIs, documentation sites used	98-99

List of Figures

1.1	Block Diagram of a Recommendation System	3
1.2	Components of a Recommendation System	6
1.3	Flow Chart to show the working of a Recommendation System	7
1.4	Phases of a Recommendation System	8
1.5	User-User KNN based Recommendation System	13
1.6	Item-Item KNN based Recommendation System	14
1.7	Matrix Factorization based Recommendation System	40

1. INTRODUCTION

1.1 Overview

MovieFlix is a small movie-catalogue and search application implemented in Java. The user interacts through a numbered console menu. Movie records are loaded from a CSV file and represented as Movie objects in an ArrayList, while a HashMap provides direct access by normalized title. The application combines basic catalogue operations with classical algorithms so that the behaviour and performance trade-offs can be observed in a practical setting.

The core data fields in the supplied implementation are Movie ID, title, language, genre, release year, director, and description. The abstract mentions actors and ratings as possible movie information, but these fields are not present in the current Movie class or CSV schema. Therefore, this report describes the implemented schema rather than claiming actor- or rating-based features.

1.2 Problem Statement

When a catalogue is stored only as an unsorted list, users may need to scan many records to find a movie. Repeated manual searching also makes it difficult to support approximate spellings, keyword queries, sorting, and record maintenance in a consistent way. The project addresses this learning problem by implementing several data structures and algorithms over one dataset, while preserving the ability to add, update, delete, save, and reload movie records.

1.3 Scope of the Project

The project is an educational, single-process command-line prototype. It does not use a database server, graphical interface, web API, user accounts, online movie metadata, actors, ratings, or concurrent execution. The advanced lab contains both implemented algorithms and conceptual explanations. It is intended to demonstrate algorithm design rather than to function as a production-scale streaming or recommendation service.

1.4 Objectives

- Build a Java console application that loads movie records from a CSV file.
- Provide exact title lookup through a HashMap and sorted title lookup through Binary Search.
- Support keyword matching using KMP and approximate title matching using Levenshtein distance.
- Sort movies by title and release year using Merge Sort.
- Provide language and genre filters and basic create, update, and delete operations.
- Persist catalogue changes through CSV save and reload operations.
- Demonstrate advanced algorithm families through M1–M6, clearly separating working code from explanatory previews.

- Document time complexity, test outcomes, limitations, and future improvements.

2.SYSTEM REQUIREMENTS

2.1 Functional Requirements

ID	Requirement	Implementation in MovieFlix
FR-01	Load catalogue	Read CSV rows into Movie objects and populate the ArrayList and title HashMap.
FR-02	Display catalogue	Print all records currently held in memory.
FR-03	Exact title search	Normalize title and look up the titleMap HashMap.
FR-04	Binary title search	Copy records, Merge Sort by normalized title, then use Binary Search.
FR-05	Keyword search	Use KMP over each movie's title plus description.
FR-06	Approximate title search	Calculate Levenshtein distance and report the best match.
FR-07	Sort catalogue	Merge Sort a copy by title or release year.
FR-08	Filter records	Search by exact language or genre substring.
FR-09	Maintain records	Add a movie, update its description, or delete a movie by title.
FR-10	Persist data	Save the in-memory catalogue to CSV or reload it from disk.
FR-11	Advanced lab	Run the M1–M6 algorithm demonstrations and explanatory previews.

2.2 Non-Functional Requirements

Requirement	Expected property / current status
Usability	Numbered menu and plain-text output; no GUI is required.
Reliability	Basic exceptions are caught around menu operations; input validation is limited.
Performance	Algorithms illustrate efficient approaches, but no formal benchmark suite is included.

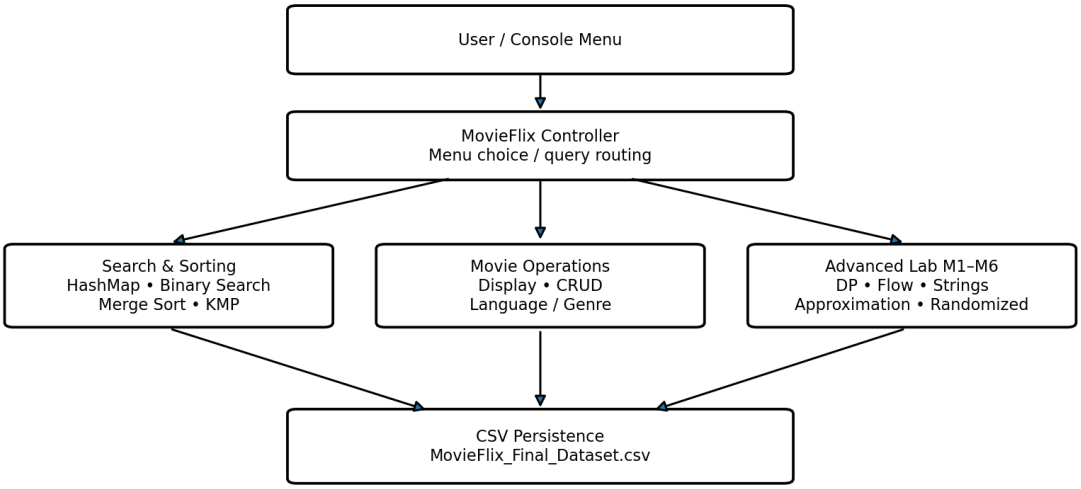
Maintainability	Core logic is grouped into static methods within one Java class; this is simple but can become difficult to extend.
Portability	Uses standard Java libraries and a relative CSV filename; a compatible JDK is required.
Data integrity	CSV save/reload is supported, but duplicate titles and malformed data need stronger validation.
Scalability	Suitable for a small educational dataset; some algorithms allocate $O(n^2)$ or exponential memory/time.

3.SYSTEM DESIGN

3.1 High-Level Architecture

The design has four logical parts: console interaction, a controller represented by the menu switch, in-memory catalogue and algorithms, and CSV persistence. The advanced menu calls the M1–M6 demonstration methods. The architecture is intentionally lightweight and does not introduce a separate service or database layer.

Figure 3.1. High-level architecture of MovieFlix



3.2 Description of each layer

The MovieFlix system is organized into four logical layers: the Presentation Layer, Application Logic Layer, Algorithm Processing Layer, and Data Storage Layer. Each layer performs a specific responsibility and works with the other layers to provide efficient movie searching, sorting, filtering, and catalogue management.

3.2.1 Presentation Layer

The Presentation Layer provides the interface through which users interact with the MovieFlix application. The current implementation uses a Java-based console interface that displays a menu of available operations and accepts user input.

The user can select options to display movie records, search for movies by title or keyword, sort records, filter movies by language or genre, and perform create, update, and delete operations. This layer displays the results generated by the application logic and presents appropriate messages when a movie is found or when an operation fails.

3.2.2 Application Logic Layer

The Application Logic Layer manages the flow of operations within MovieFlix. It receives the user's menu selection, identifies the requested operation, and invokes the corresponding method.

This layer coordinates movie searching, sorting, filtering, catalogue updates, and file operations. It also manages the in-memory movie collection and the title index used for efficient retrieval. By separating menu handling from individual operations, the application can execute different functions through a common interface.

3.2.3 Algorithm Processing Layer

The Algorithm Processing Layer contains the data structures and algorithms responsible for efficient movie retrieval and processing. It implements the core algorithms used by MovieFlix, including:

- HashMap: Provides average constant-time lookup for exact movie-title searches.
- Binary Search: Searches for a movie title in a sorted collection.
- Merge Sort: Arranges movie records alphabetically or by release year.
- Knuth–Morris–Pratt (KMP): Searches for keyword patterns in movie titles and descriptions.
- Levenshtein Distance: Identifies approximate title matches when the user enters a misspelled query.

This layer also contains the advanced DSA-3 demonstrations organized into

modules M1–M6. These include string-processing algorithms, dynamic programming, network-flow algorithms, approximation techniques, and randomized algorithms. Some advanced topics are implemented as executable demonstrations, while others are presented as explanatory previews.

3.2.4 Data Storage Layer

The Data Storage Layer is responsible for storing and retrieving movie information. MovieFlix uses a CSV file, MovieFlix_Final_Dataset.csv, as its persistent data source.

During application startup, the program reads the CSV file and loads the movie records into an ArrayList. It also constructs a HashMap using normalized movie titles to support efficient exact-title searches. When a movie is added, updated, or deleted, the application saves the modified catalogue to the CSV file.

This layer provides simple persistence without requiring a separate database server. However, the CSV file must be accessible from the application's working directory, and stronger validation would be needed to handle malformed records and duplicate titles reliably.

4. IMPLEMENTATION

4.1 Module-wise implementation

Module	Implemented in code	Explanatory preview / caveat
M1	Interval scheduling sample; Miller–Rabin call; printed query categories	No actual query classifier or API router.
M2	KMP, Z, Rabin–Karp, Aho–Corasick, suffix array, LCP	Suffix automaton/tree is preview only; suffix array is naive.
M3	Edit distances, alignment DP, matrix-chain, bitmask TSP, SOS DP	Tree diameter routine needs correction/validation.
M4	Ford–Fulkerson, Edmonds–Karp, Dinic on sample graph	Min-cut partition, matching, min-cost flow are not computed.
M5	Sample vertex-cover approximation	Reduction chains and complexity classes are explained, not solved.
M6	Miller–Rabin, randomized QuickSort, reservoir sampling, sequential sum/scan	Randomized hashing and parallel execution are explanations only.

4.2 Front-end logic (UI rendering, state management)

The front-end logic of MovieFlix is responsible for interacting with users, accepting their requests, displaying movie information, and presenting the results of different operations. It acts as the interface between the user and the application logic. Based on the project abstract, MovieFlix is implemented in Java, so the current front end is described as a console-based user interface, rather than a web-based interface.

4.2.1 UI Rendering

The user interface displays a menu containing the operations supported by the MovieFlix application. Users select an operation and provide the required input, such as a movie title, keyword, genre, or sorting preference. The application processes the request and displays the corresponding output.

The main UI operations include:

- Display Movies: Shows movie records available in the catalogue.
- Search Movies: Displays matching results using title searches, keyword searches, or other supported search methods.
- Sort Movies: Presents the catalogue in the selected order, such as alphabetical order or rating-based order, where supported by the implementation.
- Filter Movies: Displays movies matching the selected criteria, such as genre or language.
- Manage Catalogue: Provides options to add, update, or delete movie records, if enabled in the implementation.
- Display Feedback: Shows search results, operation status messages, and notifications when no matching movie is found.

The interface follows a menu-driven interaction model. After an operation is completed, the results are displayed to the user, and the menu can be shown again to allow another operation.

4.2.2 State Management

State management refers to maintaining the information required while the application is running. In MovieFlix, the main application state consists of the movie collection loaded from the dataset, the indexes used for searching, the current user selection, and the results produced by the selected operation.

The state is managed through the following components:

1. Movie Collection: Movie records are stored in an in-memory collection, such as an ArrayList, to support searching, sorting, filtering, and other catalogue operations.
2. Search Index: A HashMap can maintain normalized movie titles and their associated records or positions, allowing efficient exact-title lookup.
3. User Input State: The application reads the user's menu selection and any additional search terms or filter criteria required for the operation.
4. Operation Results: Search matches, sorted records, and filtered collections are passed to the presentation logic for display.
5. Persistent State: When catalogue modifications are supported, changes are saved to the CSV dataset so that they can be retained after the application closes.

4.2.3 Front-End Interaction Flow

The front-end interaction follows this sequence:

1. The application loads the movie dataset.
2. The user is presented with the main menu.
3. The user selects an operation and enters the required details.
4. The application logic invokes the relevant search, sorting, filtering, or catalogue-management method.
5. The processed results are returned to the interface.
6. The interface displays the results or an appropriate status message.
7. The user can choose another operation or exit the application.

4.2.4 Front-End Design Considerations

The front end is designed to keep user interaction simple and separate it from algorithm implementation. The interface accepts input and displays results, while the application and algorithm layers perform the actual processing. This separation improves readability, maintainability, and the ability to extend the application.

Input validation is also important. The application should handle invalid menu choices, empty search queries, missing movie records, and unexpected input without terminating unexpectedly.

4.3 API Integration

The MovieFlix application is designed to perform movie searching, sorting, filtering, and catalogue management using its internal Java methods and data structures. Based on the available project abstract, the application uses a CSV or text file as its data source and employs data structures such as HashMap and algorithms such as Binary Search, Merge Sort, and KMP for processing movie records.

External API integration is not specified in the available project description. Therefore, the current implementation is described as a file-based application rather than one that depends on external movie databases or web services.

4.3.1 Data Access and Integration

MovieFlix reads movie information from the local dataset, MovieFlix_Final_Dataset.csv. The records are loaded into memory and processed by the application's Java methods. The ArrayList stores movie records, while a HashMap supports efficient title-based retrieval.

The data flow follows these steps:

1. The application reads movie records from the CSV file.
2. The records are loaded into the in-memory collection.
3. The search and sorting algorithms process the requested data.
4. The results are returned to the application logic.
5. The user interface displays the results.
6. Where catalogue modifications are implemented, the updated records are saved to the local file.

4.3.2 Internal Method Integration

Instead of relying on external APIs, MovieFlix integrates its functionality through internal Java methods. The application logic invokes the appropriate method based on the user's selected operation.

Component	Integration responsibility
CSV file handling	Loads and saves movie records
ArrayList	Maintains the in-memory movie collection
HashMap	Supports efficient exact-title lookup
Binary Search	Searches an ordered collection
Merge Sort	Sorts movie records
KMP	Searches for keyword patterns
User interface	Sends requests to the application logic and displays results

4.3.3 Error Handling

The data access and integration process should handle missing dataset files, invalid records, empty search queries, and unsuccessful searches. Appropriate messages should be displayed when an operation cannot be completed. If changes are saved to the CSV file, file-writing errors should also be handled to prevent data loss.

4.3.4 Future Scope for External API Integration

MovieFlix could be extended to use an external movie information API to retrieve additional information, such as movie posters, cast details, release dates, plot summaries, and ratings. Such integration would require internet connectivity, request handling, response parsing, and suitable error handling.

5.RESULTS

This chapter presents the results obtained from the MovieFlix project. The application demonstrates the use of data structures and algorithms to manage movie records and perform searching, sorting, and filtering operations.

5.1 Movie Catalogue Display

The application loads movie records from the CSV dataset and maintains them in memory. The catalogue can be displayed to allow users to view the available movie information.

5.2 Search Results

MovieFlix supports searching for movies using title-based and keyword-based techniques. The HashMap supports efficient exact-title lookup, Binary Search can locate titles in sorted data, and KMP supports pattern matching within text.

5.3 Sorting Results

Merge Sort is used to organize movie records according to supported sorting criteria. This helps users arrange the catalogue and locate relevant movies more conveniently.

5.4 Filtering Results

The application can filter movie records according to supported attributes, such as genre or language, making it easier to identify movies that match user preferences.

5.5 Advanced DSA-3 Module Results

The advanced modules M1–M6 explore additional algorithmic concepts, including string processing, dynamic programming, graph and network-flow algorithms, approximation techniques, and randomized algorithms. Results should distinguish executable demonstrations from explanatory previews; a concept shown only as a preview should not be reported as a fully implemented or tested feature.

5.6 Summary

The MovieFlix project illustrates how data structures and algorithms can improve movie catalogue management and retrieval. Its results depend on the available dataset and the functionality actually implemented. Quantitative performance claims should be added only after conducting and recording appropriate tests.

6.Challenges & Limitations

6.1 Issues Faced During Development

During the development of MovieFlix, several challenges were encountered while organizing movie records, implementing searching and sorting algorithms, and integrating the different components of the application.

1. Managing Movie Data: Loading movie records from a CSV or text file required proper handling of data fields and record formats. Missing values or incorrectly formatted records could affect data processing.

2. Efficient Movie Searching: Different search techniques have different

requirements. For example, Binary Search requires sorted data, while exact-title searches using a HashMap require consistent handling of movie titles.

3. Implementing String Matching: Keyword searching using the Knuth–Morris–Pratt (KMP) algorithm required careful handling of pattern matching and text processing.

4. Maintaining Data Consistency: When movie records are added, updated, or deleted, the in-memory collection, search index, and saved dataset must remain consistent.

5. Understanding Advanced DSA-3 Concepts: Advanced topics across modules M1–M6 involve concepts such as dynamic programming, graph algorithms, network flow, approximation algorithms, and randomized algorithms. Integrating these concepts into a movie-management application requires additional design and testing.

6.2 Solutions Applied

The following approaches address the challenges associated with the MovieFlix design and implementation.

- Structured Data Storage: Movie records are loaded from the CSV dataset into an in-memory collection for convenient processing.
- Efficient Lookup: A HashMap is used for exact-title retrieval, while Binary Search is suitable for searching a sorted collection.
- Appropriate Sorting: Merge Sort is used to organize records according to supported sorting criteria.
- Pattern Matching: KMP is used to search for keyword patterns in text.
- Consistent Data Updates: Catalogue-management operations should update the in-memory collection and persistent CSV file consistently.
- Modular Development: Separating the user interface, application logic, algorithm processing, and data storage makes the project easier to understand, test, and maintain.
- Advanced Algorithm Demonstrations: Advanced DSA-3 concepts are presented through executable demonstrations where implemented and explanatory previews where full implementation is unavailable.

6.3 Limitations of the System

Although MovieFlix demonstrates useful data structures and algorithms, the system has certain limitations.

1. File-Based Storage: The application uses a local CSV or text file rather than a database management system. This can make concurrent access and large-scale data management difficult.
2. Data Validation: Malformed records, missing fields, and duplicate movie entries may require stronger validation.
3. Search Constraints: Binary Search requires sorted input, and HashMap performance depends on the quality of hashing and the handling of duplicate or normalized titles.
4. Limited External Integration: External movie APIs and services are not specified as part of the current implementation.
5. Advanced Module Coverage: Some DSA-3 topics are explanatory previews rather than fully implemented, tested, and integrated features.
6. Performance Evaluation: Reliable performance comparisons require systematic testing with different dataset sizes and recorded execution times.
7. User Interface: The documented application uses a Java-based console interface, which is less visually interactive than a graphical or web-based interface.

7.Conclusion & Future Work

7.1 Conclusion

MovieFlix demonstrates how data structures and algorithms can be applied to a movie catalogue management system. The project focuses on storing movie records, searching for titles and keywords, sorting records, and filtering movies according to supported criteria.
The use of ArrayList enables convenient in-memory storage, while HashMap supports efficient exact-title lookup. Binary Search, Merge Sort, and KMP

demonstrate different approaches to searching, sorting, and string matching. The project also explores advanced DSA-3 concepts through modules M1–M6. Distinguishing implemented demonstrations from explanatory previews provides a clearer understanding of the project's actual capabilities and areas that require further development.

Overall, MovieFlix provides practical experience in applying algorithmic concepts to a real-world-style application while highlighting the importance of modular design, data consistency, validation, and testing.

7.2 Future Work

The following enhancements could improve the functionality, usability, and scalability of MovieFlix.

- Graphical User Interface: Develop a graphical or web-based interface to make movie browsing and searching more interactive.
- External API Integration: Integrate a movie information API to retrieve posters, cast details, plot summaries, and other metadata.
- Database Integration: Replace or supplement CSV storage with a relational database for more reliable data management.
- Enhanced Recommendations: Implement a recommendation feature based on genres, ratings, keywords, or user preferences.
- Improved Search: Add fuzzy searching, stronger typo correction, and combined filters to improve movie discovery.
- Advanced Algorithm Implementation: Convert explanatory previews into complete, executable, and tested implementations where appropriate.
- Performance Benchmarking: Measure search and sorting performance across different dataset sizes and compare the results.
- Data Validation and Error Handling: Improve validation of user input, duplicate records, malformed data, and file operations.
- Automated Testing: Add unit tests and integration tests to verify algorithm correctness and the consistency of catalogue updates.

7.3 Skills Learned During Development

The development of MovieFlix provides opportunities to develop the following technical and professional skills:

1. Data Structures: Practical understanding of collections such as ArrayList and HashMap, including their use in storing and retrieving movie records.

2. Algorithm Design: Understanding the principles and applications of Binary Search, Merge Sort, KMP, and other algorithmic techniques.
3. Java Programming: Experience in organizing Java methods, handling user input, managing collections, and implementing application logic.
4. File Handling: Learning to read movie records from CSV or text files and persist catalogue changes where supported.
5. Problem-Solving: Identifying algorithmic requirements, selecting suitable data structures, and considering efficiency and correctness.
6. Modular Software Design: Understanding the separation of presentation, application logic, algorithm processing, and data storage.
7. Testing and Debugging: Recognizing the importance of checking input validation, algorithm behavior, data consistency, and edge cases.
8. Technical Documentation: Developing the ability to explain system architecture, implementation decisions, limitations, and possible improvements in a structured project report.

Final summary: MovieFlix combines Java programming with data structures and algorithms to demonstrate efficient movie catalogue operations. Its future development can focus on improved interfaces, stronger data management, comprehensive testing, and fuller implementation of advanced DSA-3 modules.

REFERENCES

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

2. Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.

3. Oracle. (n.d.). Java Documentation. Oracle. https://docs.oracle.com/en/java/

4. Oracle. (n.d.). Java Collections Framework. Oracle. https://docs.oracle.com/javase/8/docs/technotes/guides/collections/

5. Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. SIAM Journal on Computing, 6(2), 323–350. https://doi.org/10.1137/0206024

6. Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. Soviet Physics Doklady, 10, 707–710.

7. Kleinberg, J., & Tardos, É. (2005). Algorithm Design. Pearson.

8. MovieFlix Project Abstract. (2026–2027). MovieFlix: Movie Catalogue Management Using Data Structures and Algorithms. Project documentation prepared for the DSA-3 course.

9. MovieFlix Project Dataset. (n.d.). MovieFlix_Final_Dataset.csv. Local project dataset.

10. DSA-3 Project Documentation Guidelines. (2026–2027). Project Documentation Format and Guidelines. Course-provided materials.